

**THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF MECHANICAL ENGINEERING**



COURSE REPORT

**PBL 3: DESIGN OF MICROCONTROLLER AND SENSOR-
BASED APPLICATION DEVICE**

TOPIC: APARTMENT PARKING LOT

Supervisor: Assoc. Prof Dr. Dang Phuoc Vinh

Students: Pham Huu Phuc - Student ID: 101210087
Phan Trung Nhat - Student ID: 101210325
Vo Tan Hao - Student ID: 101210313

Class: 21CDTCLC1

Course Group: 21N05

Da Nang, May 2024

PREFACE

In recent years, with the progress of science and technology and the rapid industrialization and modernization of the country, there have been many urgent requirements and challenges posed to the intellectual community.

To continue leading the development of the increasingly prosperous country, it is necessary to invest in education and train the younger generation with sufficient knowledge to meet the growing demands of society. This requires improving the quality of education and incorporating modern teaching tools into classrooms and schools. Only with such facilities can individuals enhance their skills and meet the requirements of society.

To familiarize ourselves with the design, fabrication, and understanding of electronic components, we were assigned the project "Design and Construction of a Model Parking Lot for an Apartment Complex" by the faculty of the Electrical and Electronics Department. This project aims to reinforce our knowledge through practical application. After receiving the topic, with the guidance of Mr. Dang Phuoc Vinh and our own efforts in researching and studying relevant materials, our project has now reached a fundamental completion.

During the implementation process, despite our efforts, there may be some limitations due to time constraints and our level of expertise, which may result in some errors. Therefore, I sincerely seek the guidance, assistance, and valuable input from our professors to further enhance the quality of our project.

Contents

CHAPTER 1: INTRODUCTION	1
1.1. Problem Statement	1
1.2. Problem Survey	1
1.3. Problem Statement	2
1.4. Instructions for using the software	2
1.5. The PIC 16F877A Microcontroller	2
1.5.1. Overview of the PIC 16F877A Microcontroller	2
1.5.2. Memory Organization.....	6
CHAPTER 2: INTRODUCTION TO KEY COMPONENTS USED IN THE PROJECT.....	8
2.1. SG90 Servo Motor	8
2.2. Infrared Sensor Module:.....	9
2.3. RFID RD6300	11
2.4. LCD 1602.....	13
2.4.1. Pinout of LCD 1602	13
2.4.2. Specifications LCD1602 screen.....	14
2.4.3. Diagram and function of the pins of LCD.....	14
CHAPTER 3: CIRUIT DIAGRAM, 3D CIRUIT DESIGN, ALGORITHM FLOWCHART, AND CODE DESIGN.....	16
3.1. Hardware design:.....	16
3.1.1. Control block.....	16
3.1.2. Button block:	17
3.1.3. Motor control and sensor block	18
3.1.4. Circuit block for interfacing with a computer via the MAX232 IC ..	19
3.1.5. Power supply circuit	20
3.2. PCB Layout	21
3.3. 3D circuit diagram	22
3.4. CODE CCS.....	23
CHAPTER 4: MODEL, VISUAL INTERFACE AND USER GUIDE.....	30
4.1. Real-life product	30
4.2. Visual software, Visual interface	30
4.2.1. Login Interface	31
4.2.2. Control Interface.....	35

4.3. Instructions for use:	57
4.3.1. Hardware Instructions:.....	57
4.3.2. Software Instructions:	57
CHAPTER 5: CONCLUSION	59
RESULTS OF IMPLEMENTATION	59
5.1. Achievements	59
5.2. Development direction of the topic.....	59
REFERENCES.....	60
REFERENCE SUBJECTS	60
WEBSITE REFERENCES	60

CHAPTER 1: INTRODUCTION

1.1. Problem Statement

Currently, electronics engineering is becoming a multidisciplinary field. In addition to meeting the continuous demands from various sectors such as aerospace, telecommunications, and automation, electronics also fulfills the requirements for high-precision automated electrical testing, which has greatly assisted humans in their daily lives. Scientific and technological achievements have brought about a convenient, civilized, and modern lifestyle.

Two prominent applications of electronic technology are automatic identification and electronic alarm systems. Automatic identification technologies include barcodes, smart cards, biometrics, optical character recognition, and radio frequency identification (RFID). Electronic alarm devices include fire alarm systems, bell-based burglar alarm systems, and access control systems that utilize codes. The applications of these two technologies have significantly contributed to the safety and security of users' belongings and assets.

1.2. Problem Survey

Currently, hardware applications are increasingly common and widespread. In large city centers, we can observe the use of smart parking lots equipped with card reader modules for fully automated vehicle access control. These systems are aesthetically pleasing and convenient. Furthermore, card reader modules are extensively used in various other areas of life, such as RFID card applications for door access control, vehicle entry and cargo retrieval management, and automated toll collection stations.

Our chosen topic stems from the practical need for integrating intelligent technology into daily life. With this idea in mind, we have combined theory and practice to undertake the project "Design and Construction of a Model Residential Parking Lot."

1.3. Problem Statement

The project "Design and Construction of a Model Residential Parking Lot" presents the following requirements:

The ability to develop and expand into practical applications.

High durability and stable system operation.

Programming for PIC 16F877A microcontroller, which will be applied to the system.

Circuit with LCD display and monitoring on a computer.

Utilization of the RFID RD6300 module.

Use of the SG90 servo motor.

Incorporation of an infrared sensor module.

1.4. Instructions for using the software

- Use the CCS programming software to program and compile for the PIC 16F877A microcontroller.
- Construct a keypad consisting of 6 buttons:
 - KẾT NỐI button: Connect the COM ports.
 - NGẮT KẾT NỐI button: Disconnect the COM ports
 - LÊN button: Raise the barrier
 - XUỐNG button: Lower the barrier
 - ĐĂNG XUẤT button: Exit the current login session
 - CÓ RFID button: Detecting an RFID module that receives signals.
- Display the name and status of the barrier using the UART serial communication protocol to enable computer control and monitoring
- To power the SG90 motor, use a 12V AC/DC adapter and utilize the LM7805 circuit to step down the voltage to 5V for the microcontroller power supply

1.5. The PIC 16F877A Microcontroller

1.5.1. Overview of the PIC 16F877A Microcontroller

1.5.1.1 Summary:

- "PIC" is an acronym for "Programmable Intelligent Computer," which was originally coined by General Instrument for their first microcontroller. Microchip, a

company that acquired General Instrument, continued to develop this product and has since created nearly 100 different variations of PIC microcontrollers.

- The PIC16F887A is a popular line of PIC microcontrollers that offers a comprehensive set of features, making it suitable for a wide range of practical applications. It is considered a beginner-friendly PIC microcontroller, allowing newcomers to learn and build a foundation in PIC microcontroller programming.

- The PIC 16F877A belongs to the 16Fxxx microcontroller family and possesses the following characteristics:

- A simple programming language with 35 instructions, each with a length of 14 bits.

- All instructions are executed within one instruction cycle, except for certain branching instructions that take two instruction cycles. The instruction cycle is equal to four times the oscillator cycle.

- The Flash program memory size is 8Kx14 words, with approximately 100,000 erase/write cycles.

- The RAM memory size is 368x8 bytes.

- The EEPROM memory size is 256x8 bytes.

- Interrupt capability with up to 14 sources, including both internal and external interrupts.

- The stack memory is divided into 8 levels.

- Direct or indirect memory access using addresses

- Wide operating voltage range: 2.0V to 5.5V.

- Low power consumption: 25mA

$I_{tt} < 0.6\text{mA}$ with 5V, 4MHz

$I_{tt} = 20\mu\text{A}$ with power 3V, 32 kHz.

- There are three timer modules:

➤ Timer0: It is an 8-bit timer/counter with a prescaler.

- Timer1: It is a 16-bit timer/counter with a prescaler. It can operate even when the CPU is in SLEEP mode and can use either the oscillator or an external clock source.
- Timer2: It is an 8-bit timer/counter with both a prescaler and a postscaler.
 - There are 2 channels of Capture/Compare/Pulse Width Modulation (CCP) with 10-bit resolution.
 - There are 8 channels of 10-bit Analog-to-Digital Conversion (ADC).
 - The Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) protocols are supported by the Synchronous Serial Port (SSP), which can operate as a master for SPI and both master and slave for I2C. The USART/Synchronous Clock Line (SCL) includes a 9-bit address detection capability.
 - The Parallel Slave Port (PSP) provides 8 additional I/O lines, controlled by RD, WR, and CS signals.
 - Due to time constraints, we focused on exploring the features of the PIC 16F877A that are relevant to our project. Here are some features of the PIC 16F877A that are applied in our project:
 - Memory organization of PIC 16F877A.
 - Functions of Port I/O.
 - Functions and configuration of Timer 0.
 - Functions and configuration of Pulse Width Modulation (PWM).
 - Definition of interrupts, interrupt sources, and in-depth understanding of timer interrupts and external interrupts are two functionalities used in this project.

1.5.1.2 Pinout diagram and block diagram of PIC 16F877A:

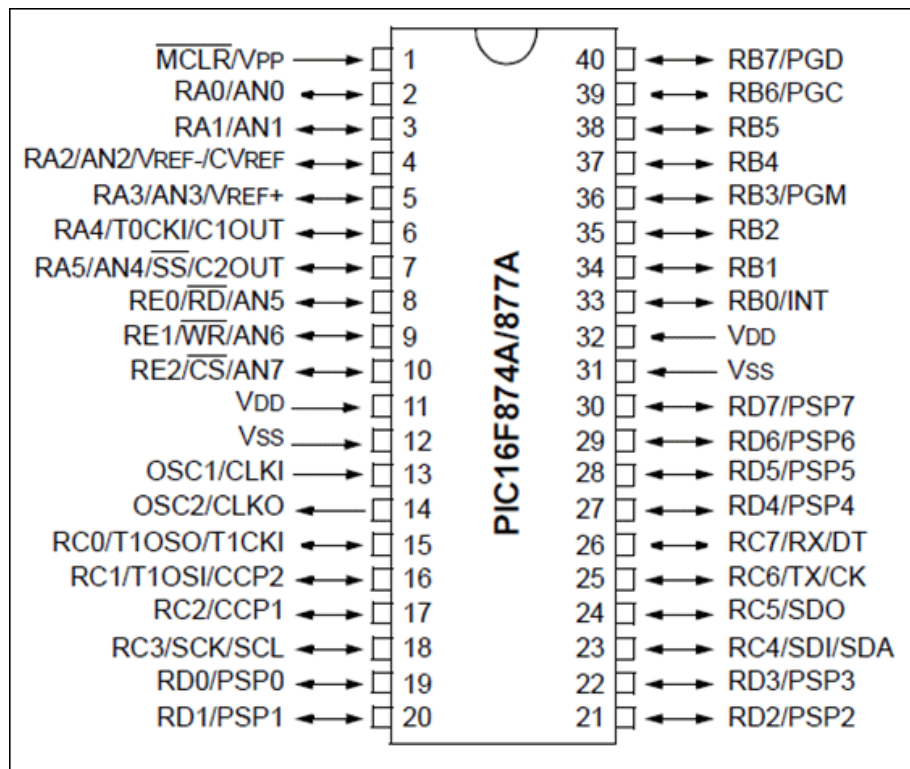


Figure 1. Pinout diagram of PIC 16F877A.

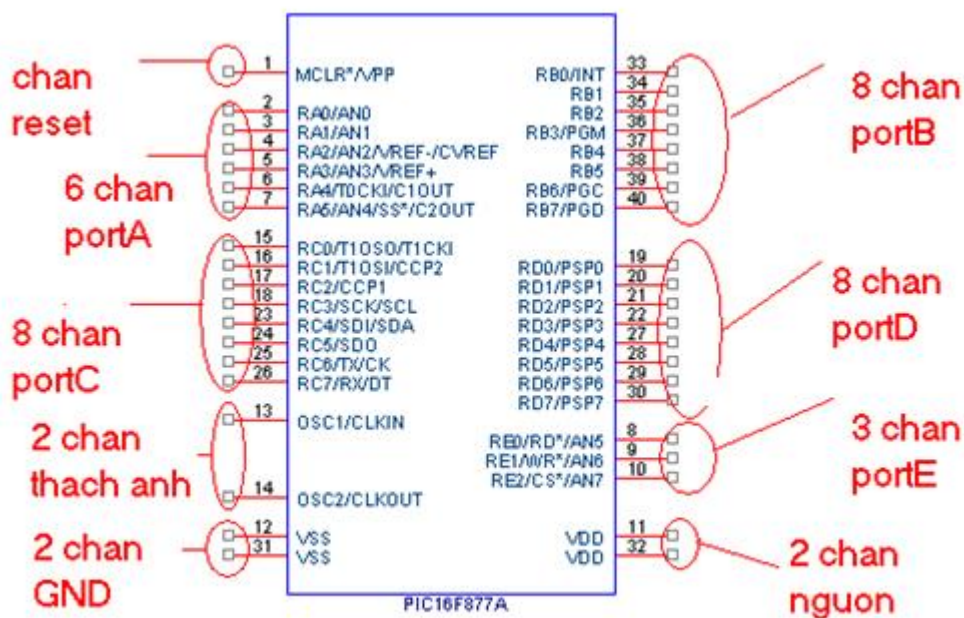


Figure 2. Block diagram of the Ports of PIC 16F877A.

1.5.1.3 Comments:

- From the pin diagram and block diagram above, we can make the following initial observations: The PIC16F877A has a total of 40 pins, which are divided into 5 ports, 2 power pins, 2 ground pins, 2 crystal pins, and one reset pin.

- The 5 ports of the PIC16F877A are:

- PORT B: 8 pins
- PORT D: 8 pins
- PORT C: 8 pins
- PORT A: 6 pins
- PORT E: 3 pins

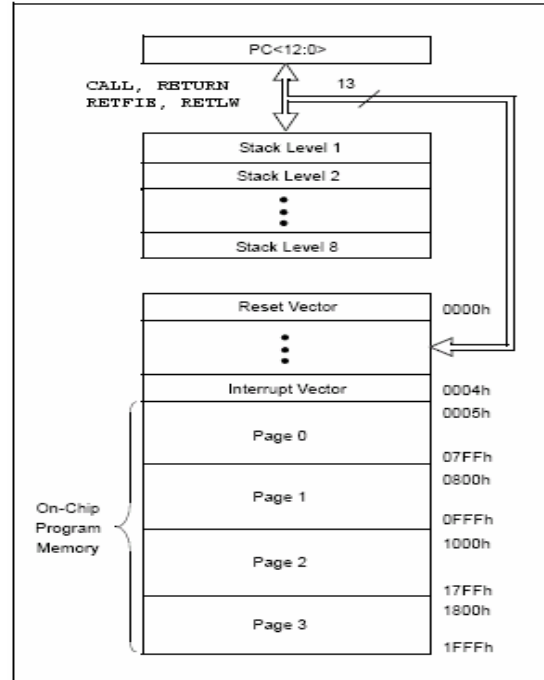


Figure 3. Structure of the program memory in PIC 16F877A.

1.5.2. Memory Organization

- The memory structure of the PIC16F877A microcontroller includes the program memory and data memory.

1.5.2.1 Program Memory:

- The program memory of the PIC16F877A is flash memory with a capacity of 8K words (1 word = 14 bits) and is divided into multiple pages (from page 0 to page 3). Therefore, the program memory can hold $8 * 1024 = 8192$ instructions (since each instruction, after encoding, has a size of 1 word or 14 bits). To address the 8K word program memory, the program counter has a capacity of 13 bits (PC<12:0>).
- When the microcontroller is reset, the program counter points to address 0000h (Reset vector). When an interrupt occurs, the program counter is directed to address 0004h (Interrupt vector). The program memory does not include stack memory and is not addressed by the program counter.

1.5.2.2 Data Memory:

- The data memory of the PIC16F877A is divided into 4 banks, with each bank having a capacity of 128 bytes.
- Unlike the separate division of the 8051's data memory into two distinct banks (the first 128 bytes belonging to bank 1 as general-purpose RAM and the remaining 128 bytes belonging to bank 2 as special function registers or SFRs), the 4 banks of data memory in the PIC16F877A are organized differently.
- Each bank of the PIC16F877A's data memory includes both special function registers (SFRs) located in the lower memory addresses and general-purpose registers (GPRs) located in the remaining memory space of each bank. The memory space for these general-purpose registers is where users store data during program execution. It is recommended to declare and store all data variables within this memory space.
- In the structure of the PIC16F877A's data memory, SFRs that are frequently used (such as the STATUS register) are placed in all banks for convenient access. This is done because accessing a particular register requires specifying the bank containing that register. By placing commonly used registers in all banks, it simplifies the access process and reduces the number of program instructions needed.

CHAPTER 2: INTRODUCTION TO KEY COMPONENTS USED IN THE PROJECT

2.1. SG90 Servo Motor

This is the motor used in the project:

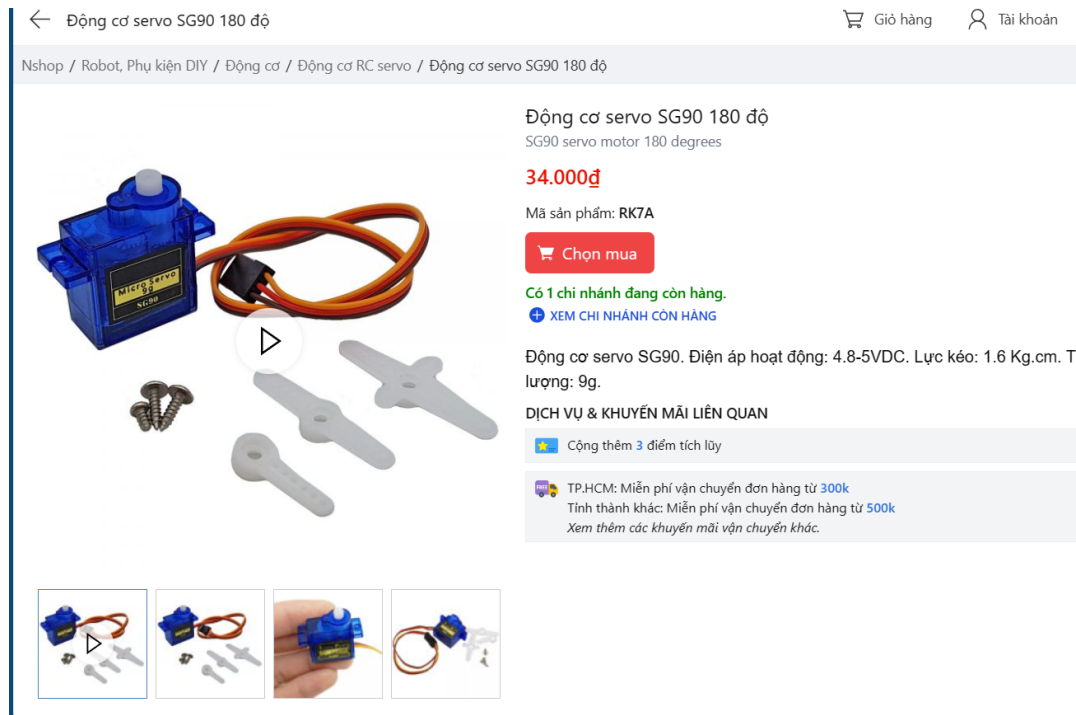


Figure 4: SG90 Servo Motor

SG90 Servo Motor (180 Rotation Angle) is a popular servo used in small and simple control models such as robotic arms. The motor has fast response speed, built-in motor driver, easy to control the rotation angle by PWM pulse width modulation method.

Specifications:

- Weight: 9g
- Dimensions: 22.2x11.8.32 mm
- Torque: 1.8kg/cm
- Operating speed: 60 degrees in 0.1 seconds
- Operating voltage: 4.8V(~5V)
- Operating temperature: 0 C - 55 C

- Connect the red wire to 5V, the brown wire to the ground, the orange wire to the pulse generator pin of the microcontroller. At the pulse pin, provide a pulse from 1ms-2ms to control the desired rotation angle.

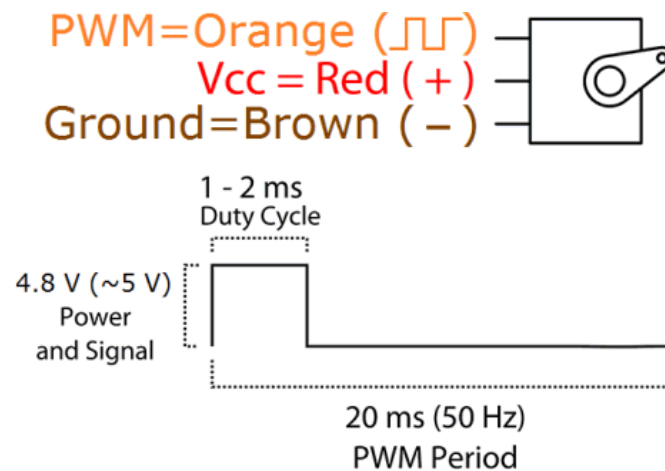


Figure 5: Wirng diagram

2.2. Infrared Sensor Module:

This is the Module used in the project:

Nshop / Cảm biến / Cảm biến khoảng cách vật cản / Cảm biến vật cản hồng ngoại / Cảm Biến Vật Cản Hồng Ngoại

Cảm Biến Vật Cản Hồng Ngoại
IR Infrared Obstacle Avoidance Sensor Module

10.000đ

Mã sản phẩm: FM52

[Chọn mua](#)

[XEM CHI NHÁNH CÒN HÀNG](#)

Cảm biến vật cản hồng ngoại bộ so sánh sử dụng LM393, làm việc ổn định, điện áp làm việc: 3.3V – 5V DC, kích thước: 3.2cm * 1.4cm

DỊCH VỤ & KHUYẾN MÃI LIÊN QUAN

[Cộng thêm 1 điểm tích lũy](#)

[TP.HCM: Miễn phí vận chuyển đơn hàng từ 300k](#)
Tỉnh thành khác: Miễn phí vận chuyển đơn hàng từ 500k
[Xem thêm các khuyến mãi vận chuyển khác.](#)

Figure 6: Module infrared sensor

- The infrared obstacle sensor is capable of adapting to the environment and consists of a transmitter and a receiver pair. The infrared beam is emitted at a specific frequency, and when it detects an obstacle in the transmission path (reflective surface), it reflects back to the infrared receiver. After comparison, the green LED will light up, and at the same time, it provides a low-level digital output signal.
 - The effective working distance of the infrared obstacle sensor is 2 to 5cm, and the operating voltage ranges from 3.3V to 5V. The sensitivity of the sensor can be adjusted using a potentiometer. The sensor is easy to assemble, user-friendly, and convenient to use.
 - The infrared obstacle sensor can be widely used in applications such as obstacle avoidance in robots, obstacle detection in vehicles, and line following systems.
 - Technical specifications:
 - The sensor module uses the LM393 comparator, which provides stable operation.
 - Operating voltage: 3.3V - 5V DC.
 - When powered on, the power indicator LED lights up in red.
 - It has a 3mm screw hole for easy fixation and installation.
 - Dimensions: 3.2cm * 1.4cm.
 - The voltage threshold of the modules has been adjusted through potentiometers.
- If used in normal mode, please do not adjust the potentiometers arbitrarily.

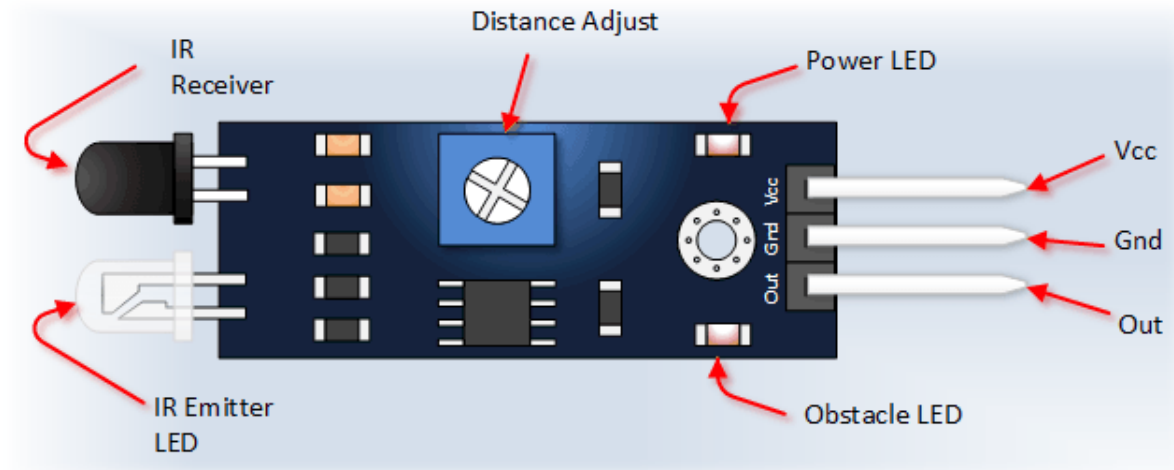


Figure 7: This is the basic wiring diagram for the infrared sensor module

The communication ports of the infrared sensor module include:

- VCC: Voltage converted from 3.3V to 5V (can be directly connected to the 5V and 3.3V pins of the microcontroller).
- GND: External ground connection.
- OUT: Digital output (0 and 1) signal.

2.3. RFID RD6300

Pin	Description
1	This is the TX pin. TX is a data transmit pin of UART of RFID reader
2	This is the RX pin which is a data receive pin
3	This pin is not used
4	This is the GND pin
5	This is the 5V power supply pin

Table 1: Wiring diagram

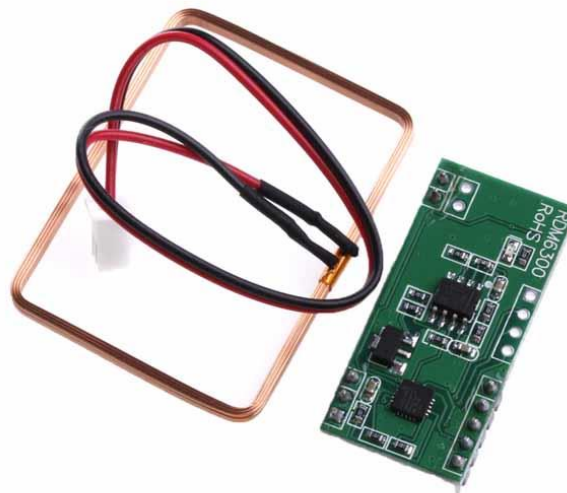


Figure 8: RFID RD6300

Pin configuration of RDM6300 RFID reader module

The RDM6300 module consists of 10 pins out of which only 5 pins are required to interface this RFID tag reader with Arduino. You can view the pinout below:

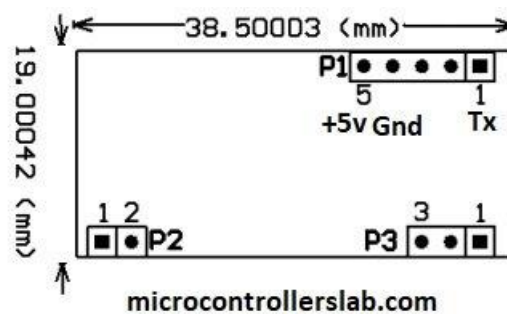


Figure 9: Wiring diagram

The RDM6300 module consists of three headers as shown above. Details of P1 header pins are given below:

Header 1 Pins Description

The antenna is connected to the P2 header. Details of P2 header pins are given below:

- Pin number one is terminal number one of Antenna
- Pin number two is terminal two of Antenna.

Pin number one on header P3 is a **light emitting diode**. It is a status or indication LED. It normally remains on and turns off while reading the RFID tag or card.

Below you can view each individual pin in a clear manner:

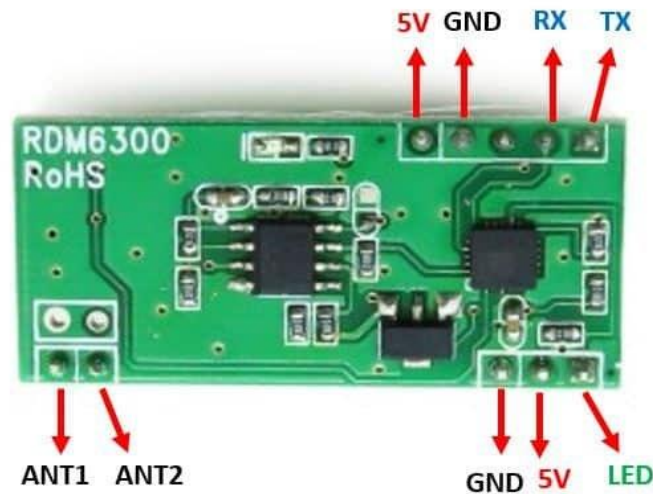


Figure 11: Wiring diagram

2.4. LCD 1602

LCD1602 text screen using HD44780 driver, capable of displaying 2 lines with 16 characters each, the screen is durable, very popular, lots of sample code and easy to use, suitable for beginners and beginners project.

The display is easy to communicate and displays parameters according to the ASCII code table. It is also possible to add information display with custom characters.

2.4.1. Pinout of LCD 1602



Figure 12: LCD 1602

2.4.2. Specifications LCD1602 screen

<i>Datasheet</i>	<u>LCD1602</u>
<i>pins</i>	16
<i>Operating voltage</i>	5V
<i>Drive</i>	HD44780
<i>Number of display cells</i>	32 display memory cells
<i>Background</i>	Green and blue
<i>Size</i>	80 x 36 x 12.5 mm

Table 2: Function of LCD pins

2.4.3. Diagram and function of the pins of LCD

LCD has many types and their pin numbers are also different, but there are 2 common types are 14 pins and 16 pins. The difference is the power supply pins for the lamp background, and the control pins remain unchanged.

pin	Symbol	Function
1	VSS	Ground pin for LCD, when designing the circuit, we connect this pin to GND of the control circuit
2	VDD	Power supply pin for LCD, when designing the circuit, we connect this pin to VCC=5V of the control circuit
3	VEE	Adjust LCD Contrast
4	RS	Register select pin. Connect RS pin to logic "0" (GND) or logic "1" (VCC) to select register
5	RW	Read/write mode selector pin (Read/Write). Connect the R/W pin to logic "0" to make LCD work in write mode, or connect to logic "1" to make LCD read mode.
6	E	Enable pins (Enable). After the signals are placed on the DB0-DB7 bus, commands are accepted only when

		there is an enable pulse of the E pin.
7-14	D0-D7	The eight lines of the data bus are used to exchange information with MPU.
15	A	Positive source for backlight
16	K	GND for backlight

Table 3: Function of LCD pins

In 16 pins of LCD are divided into 4 types of signals as follows:

Power supply pins: Pin 1 is ground pin (0V), the second pin is Vdd connected to +5V source. The 3rd leg used to adjust contrast is usually connected to the rheostat, adjust until you see the character, then stop, in the practice set, it's already adjusted.

Control pins: Pin 4 is the RS pin used to control register selection. The R/W pin is used to control the read and write process. The E pin is the pin pulse enable pin.
Data pins D7÷D0: Pins 7 to 14 are 8 pins used to exchange data between the control device and LCD.

LED_A and LED_K pins: Digital pins 15, 16 are 2 pins used to power

Backlight for visibility at night.

CHAPTER 3: CIRCUIT DIAGRAM, 3D CIRCUIT DESIGN, ALGORITHM FLOWCHART, AND CODE DESIGN

3.1. Hardware design:

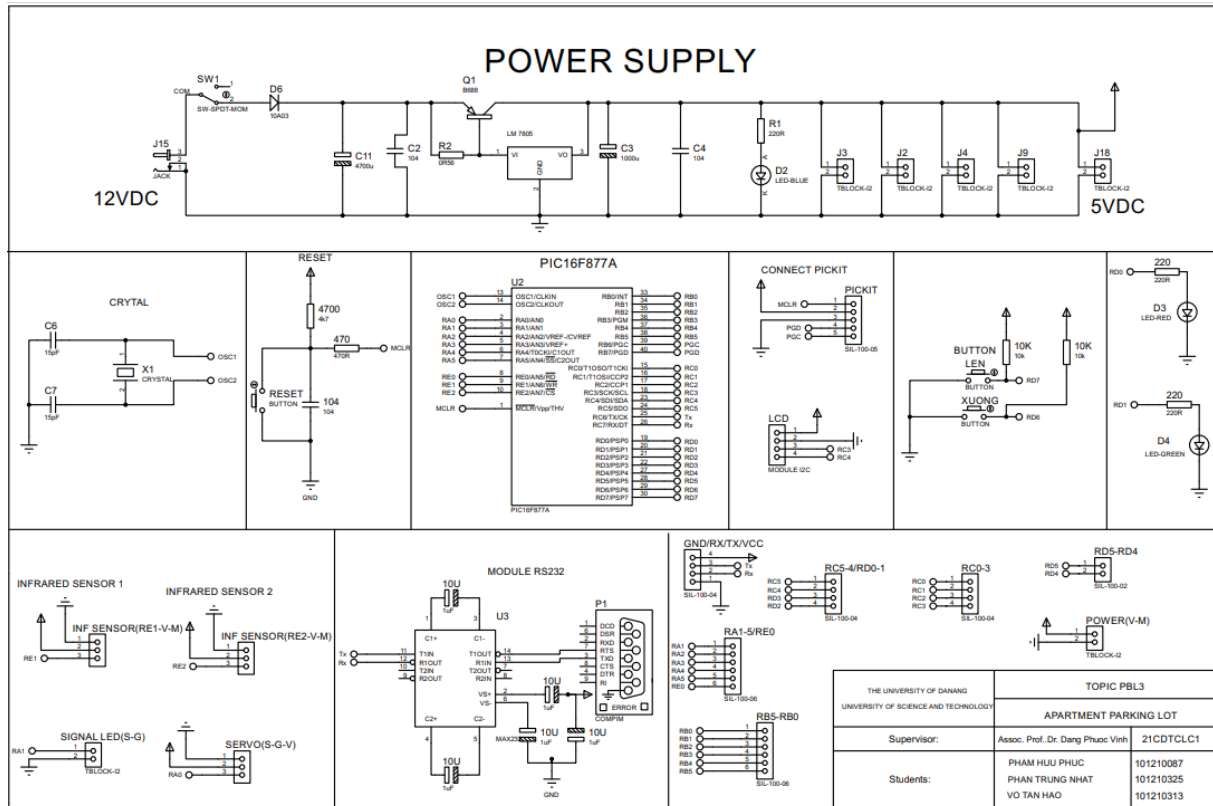


Figure 13: Schematic diagram

3.1.1. Control block

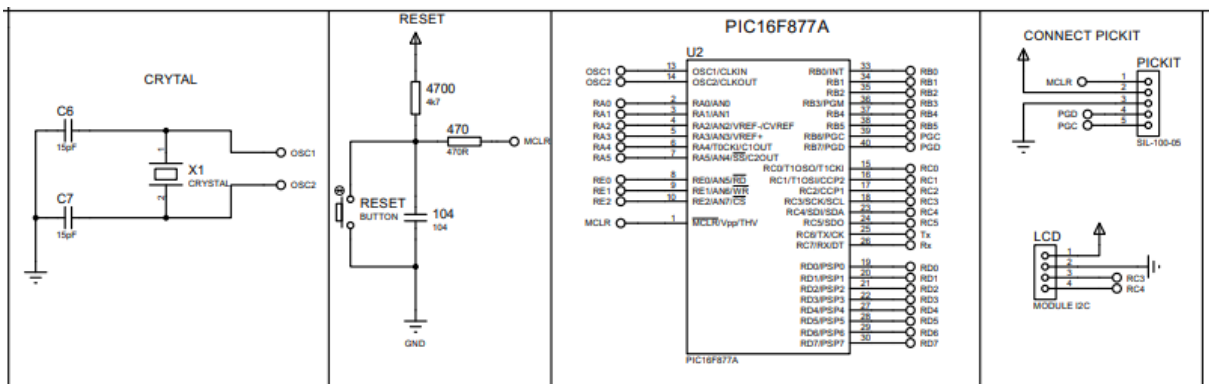


Figure 14: Control block diagram

- The function of the components in the circuit:
- Quartz crystal (thạch anh): It provides the clock signal for the microcontroller,

allowing it to synchronize its operations.

- Reset: The reset component is used to reset the microcontroller, bringing it back to its initial state or restarting its operation.
- Pickit connection: The Pickit connection allows for programming and debugging of the microcontroller. It is used to connect an external programmer/debugger tool to the microcontroller.
- 2 LED lights: The LED lights (D0 and D1) are used for testing the connectivity of the circuit. They can be controlled by the microcontroller to indicate certain states or conditions.
- PIC 16F877A: This is the microcontroller itself, which is the central processing unit of the circuit. It executes the program instructions and interacts with other components based on the input signals and programmed logic.
- Button1: This button is used for resetting the microcontroller. When pressed, it triggers a reset signal to restart the microcontroller's operation.
- J1 connector: The J1 connector is used for programming the microcontroller. It provides a connection interface for the programmer to upload the code into the microcontroller's memory.
- Quartz crystal X1 and capacitors C1, C2: These components, along with the quartz crystal, create an oscillation circuit that generates a stable clock signal for the microcontroller. The capacitors help in filtering noise and stabilizing the oscillation frequency.
- These components work together to provide the necessary functionality for the circuit, including clock synchronization, reset capability, programming interface, testing indicators, and microcontroller operations.

3.1.2. Button block:

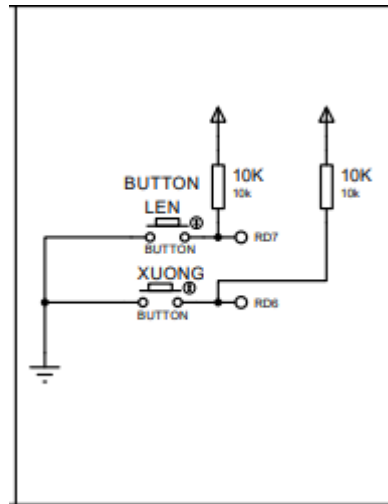


Figure 15: Button block diagram

- The button block consists of two sequentially operated buttons: LEN (up) and XUONG (down).
- LEN (up): When the LEN button is pressed, it triggers the action of rotating the barrier by 90 degrees in the clockwise direction. This button is associated with the upward movement or rotation of the barrier.
- XUONG (down): When the XUONG button is pressed, it triggers the action of rotating the barrier back to its initial position or 0 degrees. This button is associated with the downward movement or rotation of the barrier.
- These buttons provide a user interface to control the barrier's movement. The specific implementation and functionality of these buttons will depend on the circuit design and the programming logic associated with them. When the buttons are pressed, the microcontroller or other connected components detect the change in electrical state and execute the corresponding actions to control the barrier's rotation.

3.1.3. Motor control and sensor block

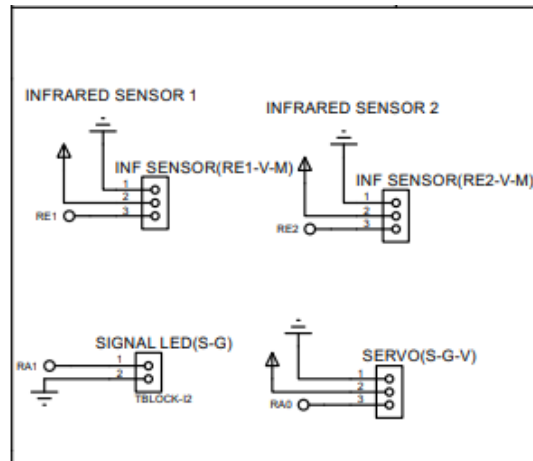


Figure 16: Motor control and sensor diagram

3.1.4. Circuit block for interfacing with a computer via the MAX232 IC

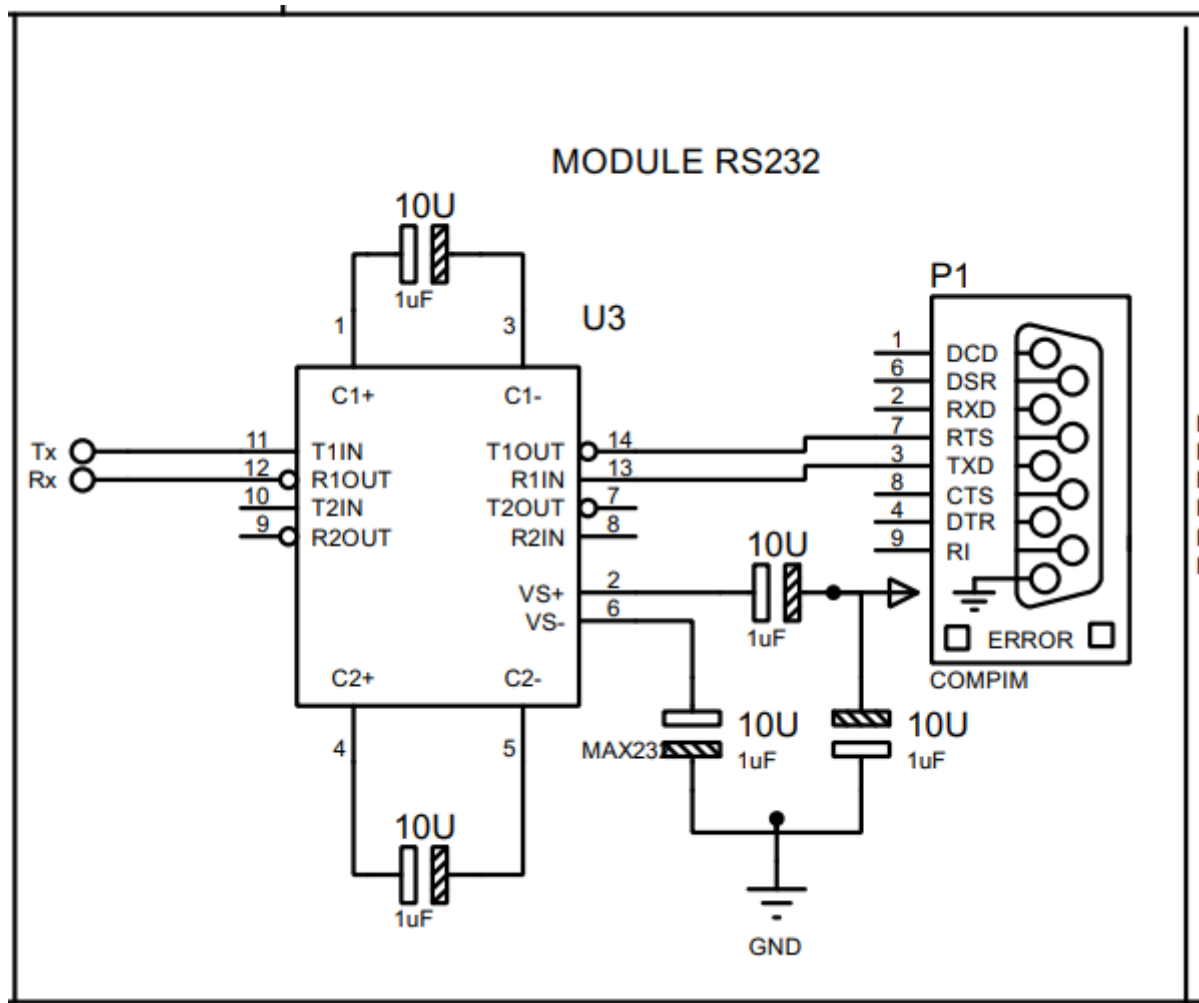


Figure 17: RS232 diagram

The RS-232 signal standard defined by the Electrical Industries Association (EIA)

specifies that a logic "1" corresponds to voltage levels ranging from -3VDC to -15VDC, while a logic "0" corresponds to voltage levels ranging from 3VDC to 15VDC. However, microcontroller chips typically use logic levels based on the TTL standard, where a logic "0" corresponds to voltage levels from 0VDC to 0.8VDC, and a logic "1" corresponds to voltage levels from 2.2VDC to 5VDC. Therefore, to facilitate communication between a microcontroller and a computer via the COM port, users need to use an interface circuit to convert between these two signal standards. In this case, the MAX232 IC is used for this signal conversion purpose.

3.1.5. Power supply circuit

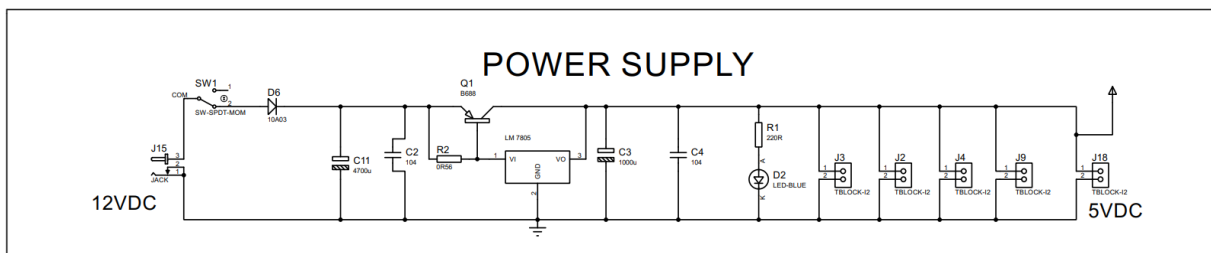


Figure 18: Power supply diagram

The power supply circuit includes various components to ensure proper functionality and protection. Here are some specific components mentioned in your description:

- Diode for reverse polarity protection: A diode is used to prevent reverse polarity connection, which can cause damage to the circuit. It allows current to flow only in one direction, protecting the circuit from incorrect power connections.
- Capacitors for stabilization and noise filtering: Capacitors are used to stabilize the voltage and filter out any noise or fluctuations in the power supply. They help provide a steady and clean power output to the connected components.
- 7805 voltage regulator: The 7805 is a commonly used linear voltage regulator that reduces the input voltage to a regulated 5V output. It ensures that the voltage supplied to the VDK (presumably a component or module) remains constant and within its specified operating range.
- TBLOCK 12V for motor supply: The TBLOCK (presumably a terminal block or connector) labeled 12V is used to provide power specifically to the motor. It supplies the necessary 12V voltage to operate the motor.

- 5 TBLOCK 5V for backup power: The 5 TBLOCK (another terminal block or connector) labeled 5V is used as a backup power source. It provides a redundant or secondary 5V power supply, which can be used if the primary power source fails or is unavailable.

3.2. PCB Layout

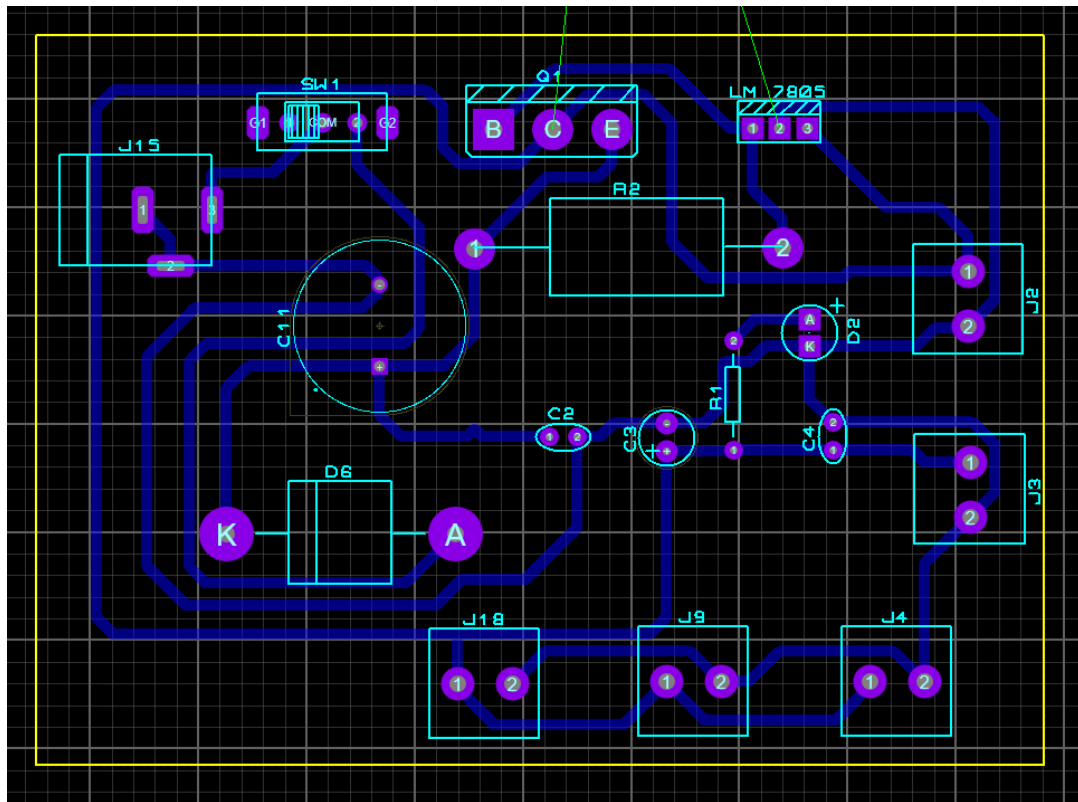


Figure 19: PCB Layout Power circuit

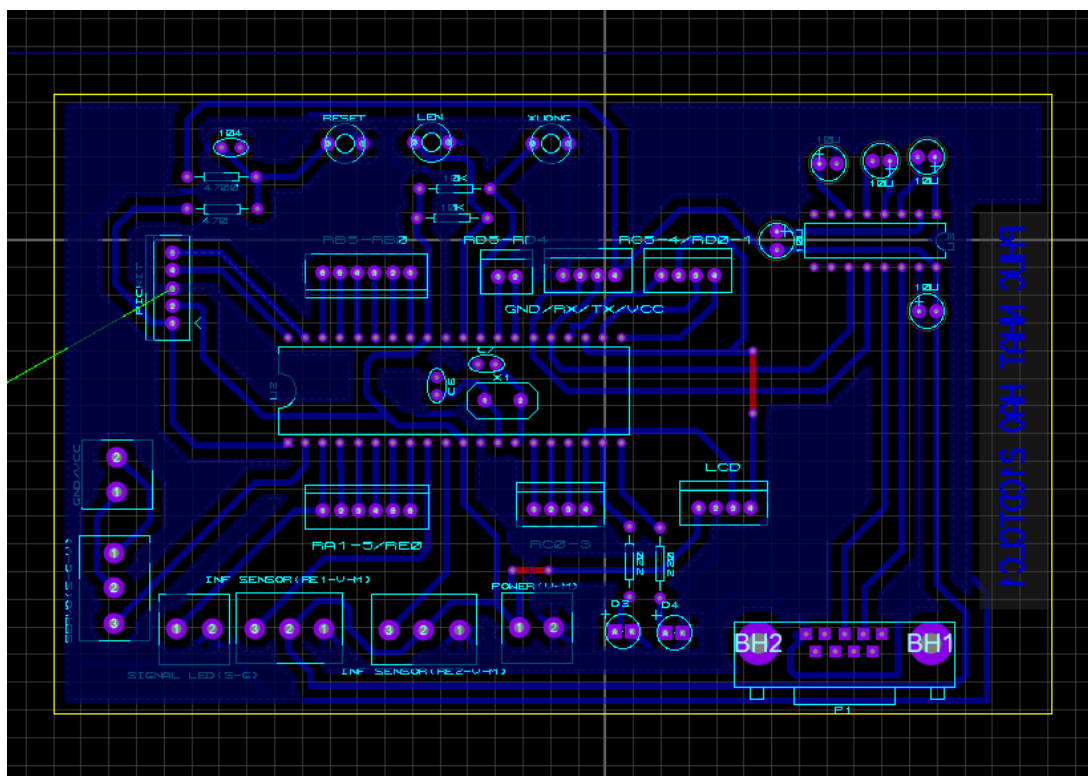


Figure 20: PCB PIC circuit

3.3. 3D circuit diagram

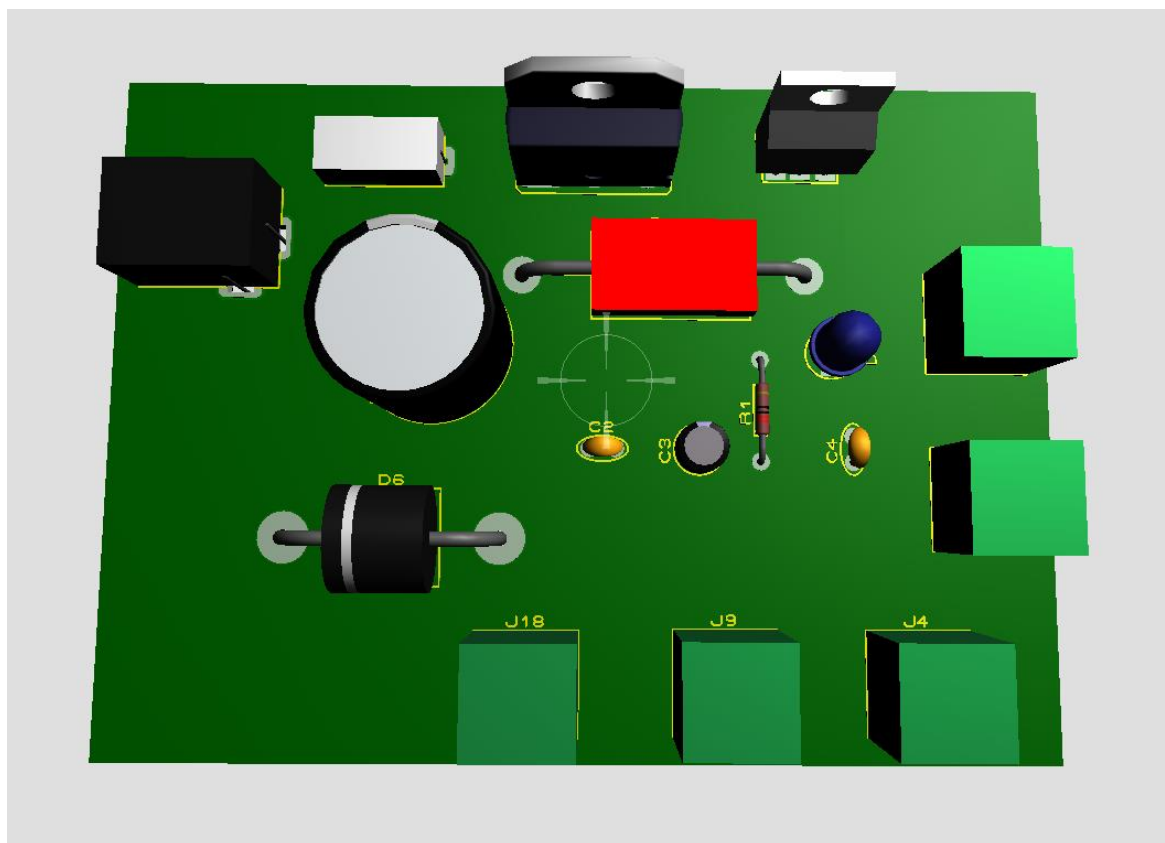


Figure 21: 3D Power circuit

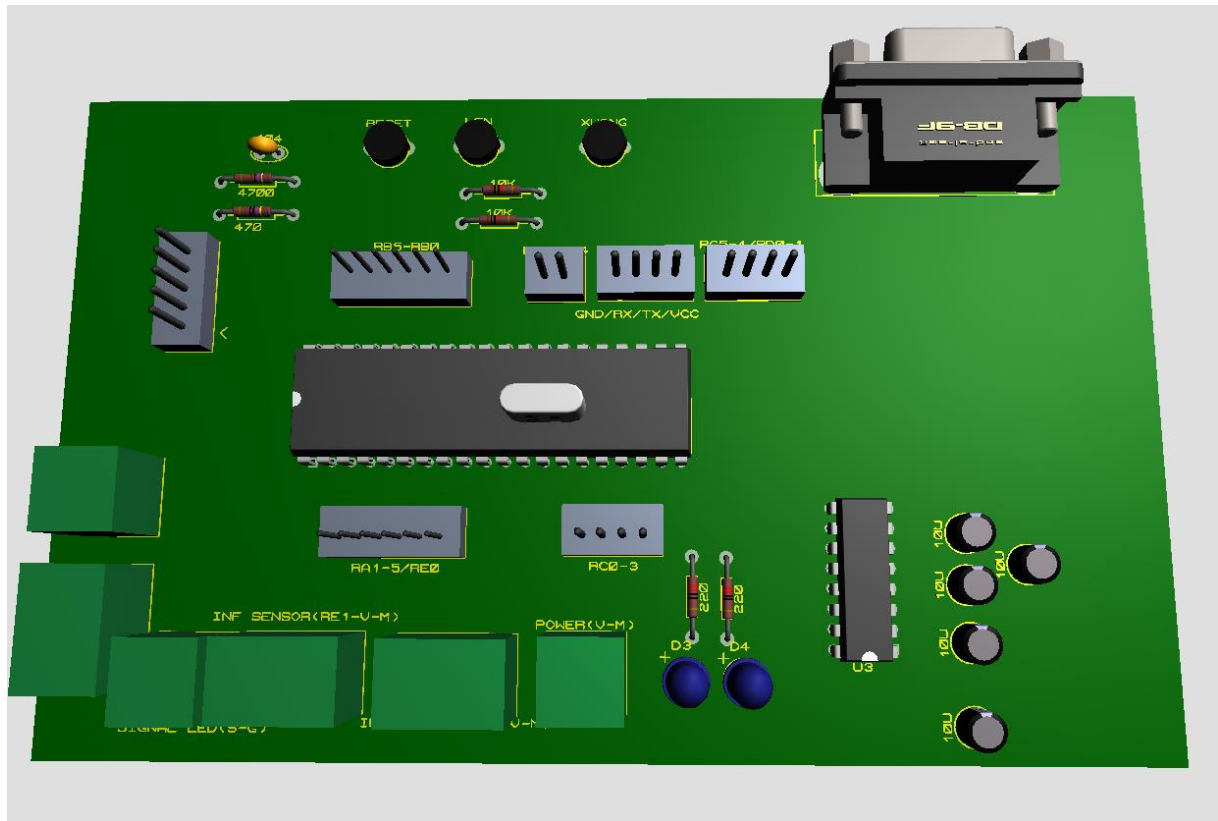
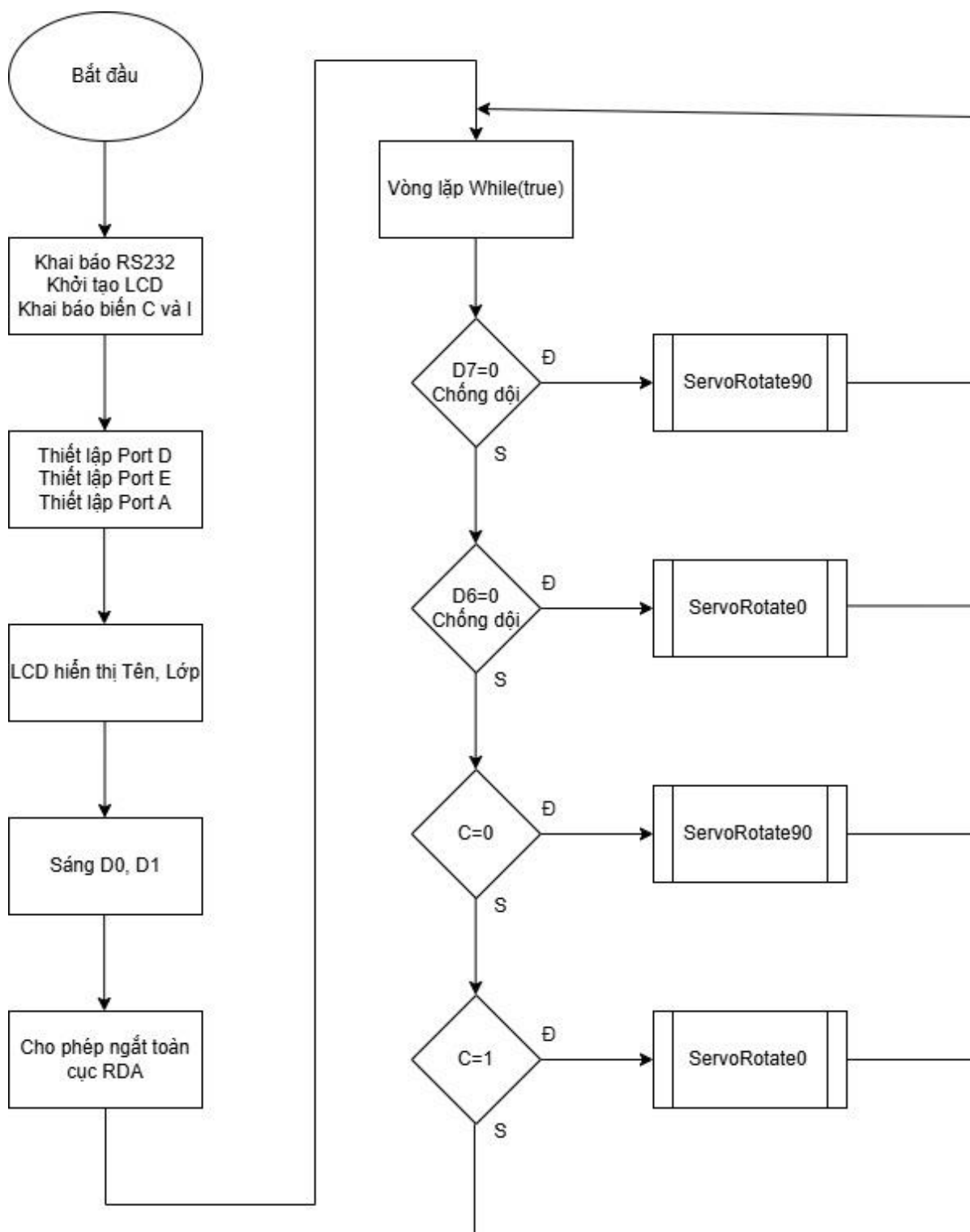
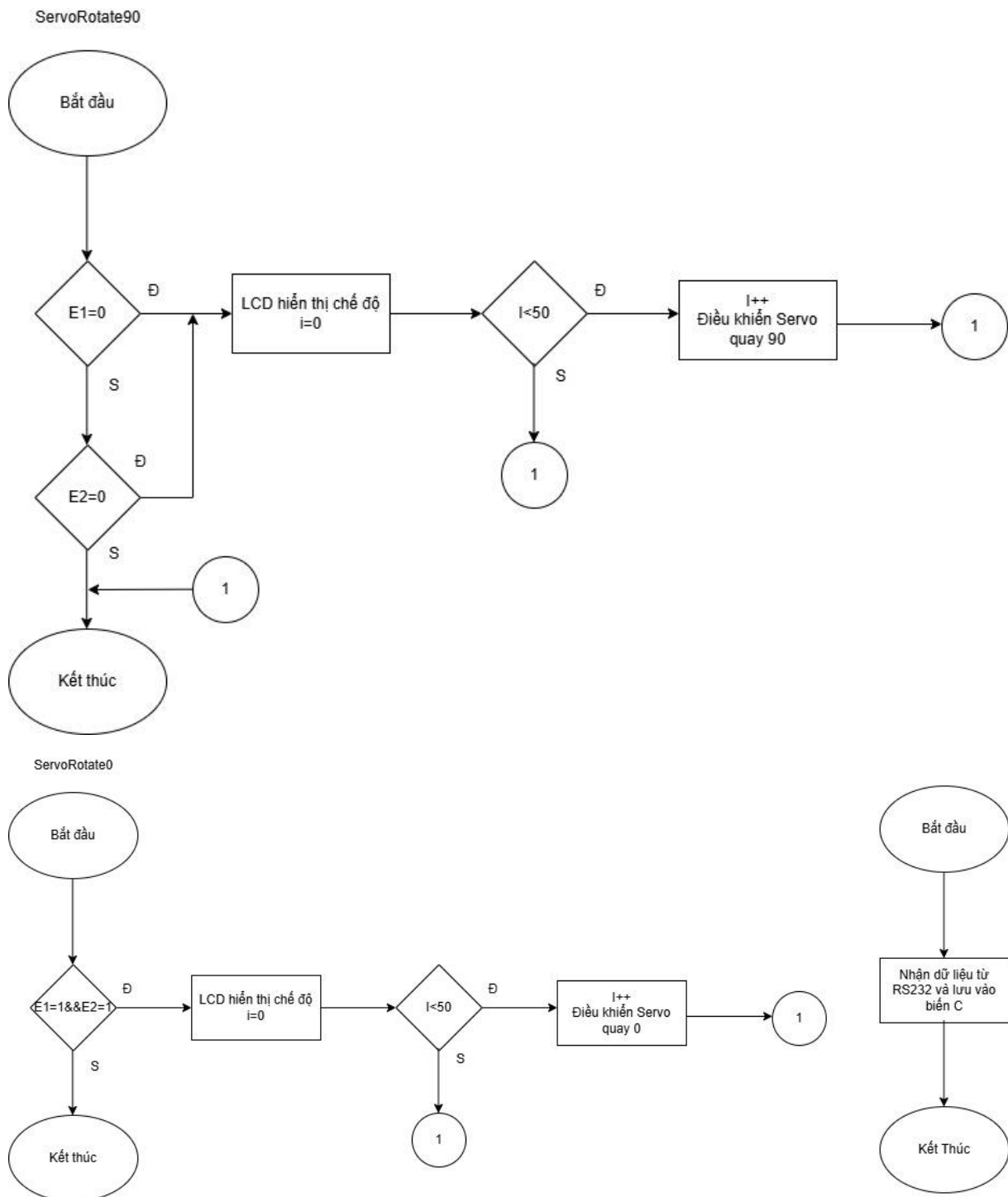


Figure 22: 3D Pic circuit

3.4. CODE CCS

Lưu đồ thuật toán:





CODE CCS:

```
#include <test.h>
```

```
#use rs232(baud=9600,parity=N,xmit=pin_c6,rcv=pin_c7,bits=8)
```

```
#use i2c(Master,Fast=100000, sda=PIN_C4, scl=PIN_C3,force_sw)
```

```
#include "i2c_Flex_LCD.c"
```

```
char c;
```

```
int i;
```

```
void servoRotate0()
```

```
{
```

```
if (input(PIN_E1)==1 && input(PIN_E2)==1)
```

```
{
```

```
    lcd_clear();
```

```
    lcd_gotoxy(5,1);
```

```
    printf(lcd_putc, "STATUS");
```

```
    lcd_gotoxy(6,2);
```

```
    printf(lcd_putc, "CLOSE");
```

```
    for(i = 0; i<50; i++)
```

```
    {
```

```
        output_high(PIN_A0);
```

```
        delay_us(1500);
```

```
        output_low(PIN_A0);
```

```
        delay_us(18500);
```

```
    }
```

```
}
```

```
}
```

```
void servoRotate90()
```

```
{  
if (input(PIN_E1)==0 || input(PIN_E2)==0)  
{  
    lcd_clear();  
    lcd_gotoxy(5,1);  
    printf(lcd_putc, "STATUS");  
    lcd_gotoxy(6,2);  
    printf(lcd_putc, "OPEN");  
    for(i = 0; i<50; i++)  
    {  
        output_high(PIN_A0);  
        delay_us(2300);  
        output_low(PIN_A0);  
        delay_us(17700);  
    }  
}  
}
```

```
#int_rda
```

```
void noi()
```

```
{
```

```
    c =getch();
```

```
}
```

```
void main()
```

```
{
```

```
set_tris_D(0b00000011);
```

```
output_high(PIN_D0);
```

```
output_high(PIN_D1);
set_tris_E(0);
lcd_init(0x4E,16,2);
set_tris_A(0);//output_high(PIN_D1);
lcd_gotoxy(2,1);
printf(lcd_putc, "PHUC NHAT HAO");
lcd_gotoxy(4,2);
printf(lcd_putc, "21CDTCLC1");
enable_interrupts(GLOBAL);
enable_interrupts(INT_RDA);
delay_ms(50);
while(TRUE)
{
    if (input(PIN_D7) == 0)
    {
        delay_ms(50);
        if (input(PIN_D7) == 0)
        {
            servoRotate90();
            delay_ms(1);
        }
    }
    if (input(PIN_D6) == 0)
    {
        delay_ms(50);
        if (input(PIN_D6) == 0)
        {
            servoRotate0();
            delay_ms(1);
        }
    }
}
```

```
    }  
    }  
    if (c == '1')  
    {  
        servoRotate90();  
        delay_ms(1);  
    }  
    if (c == '2')  
    {  
        servoRotate0();  
        delay_ms(1);  
    }  
  
}  
  
}
```

CHAPTER 4: MODEL, VISUAL INTERFACE AND USER GUIDE

4.1. Real-life product

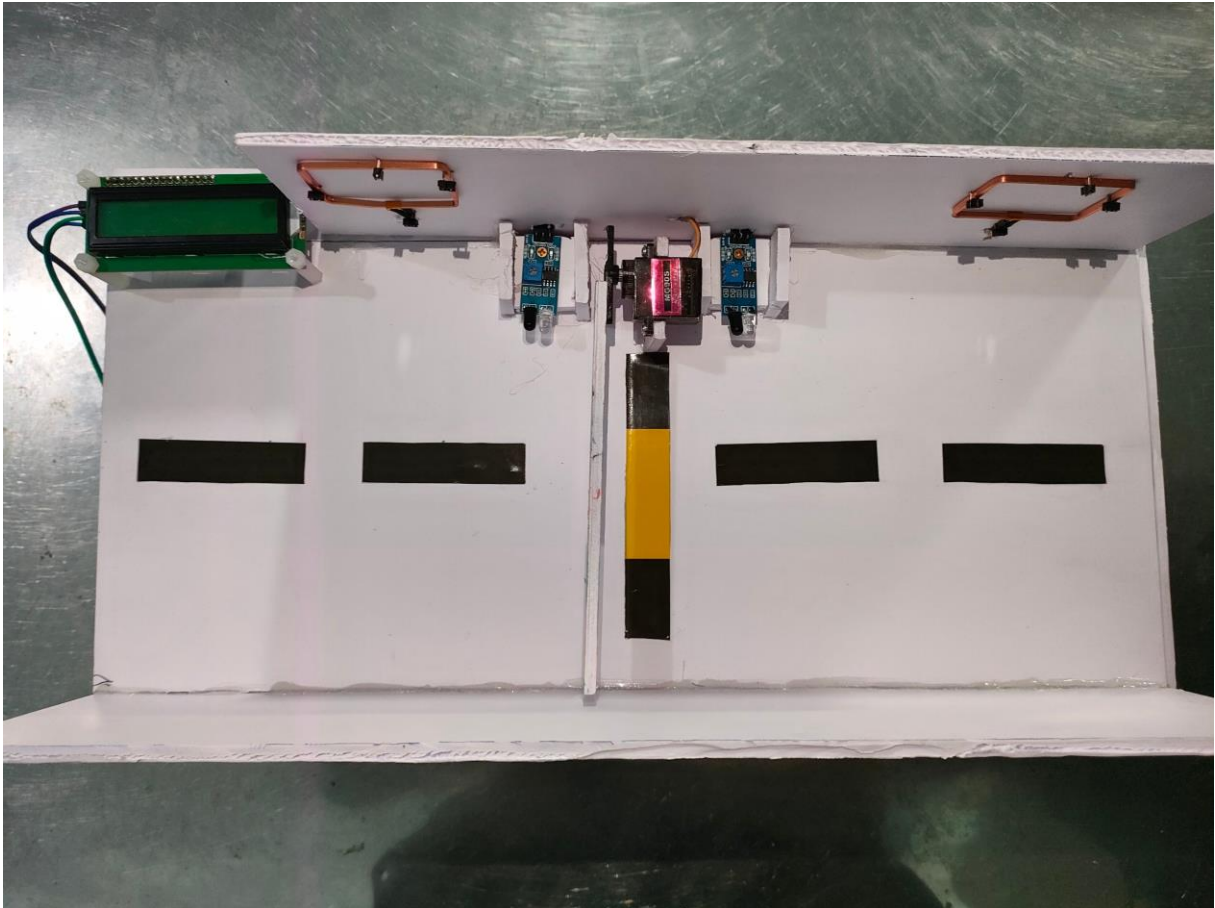


Figure 15: Actual model

4.2. Visual software, Visual interface



Figure 16. Visual software 2019

- Visual Studio 2019 is one of the most popular integrated development environments (IDEs) used for developing applications for various platforms such as Windows, web, mobile, and cloud. Developed by Microsoft, it provides a range of tools, compilers, and resources to help developers build applications from simple to complex.
- Below are some highlights of Visual Studio 2019:
 - Multi-language and platform support: Visual Studio 2019 supports application development in multiple programming languages such as C#, C++, Visual Basic, F#, JavaScript, Python, and more. It also allows developing applications for multiple platforms like Windows, Android, iOS, web, and cloud.
 - Powerful tools: Visual Studio 2019 comes with a range of powerful tools such as code editor, debugger, compiler, source code manager, and many other features to help developers increase productivity.
 - Support for new technologies: Visual Studio 2019 is continuously updated to support the latest technologies and provides project templates and utilities for developing applications based on new trends like artificial intelligence (AI), machine learning, and Internet of Things (IoT).
 - Collaboration and cloud service integration: Visual Studio 2019 integrates well with Microsoft cloud services such as Azure for easy application development and deployment. It also provides team collaboration tools that allow developers to work together on projects.
 - Community and support: Visual Studio 2019 has a large community with many resources such as documentation, forums, and community-driven tools, making learning and problem-solving easier.

4.2.1. Login Interface

DANG_NHAP

Thursday, May 2, 2024

WELLCOME TO SYSTEM SMART PARKING

Get Started Form Login Below

USER

PASSWORD

LOGIN

CODE:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace BAIDOXETUDONG
{
    public partial class FormDANG_NHAP : Form
    {

```

```
string tentaikhoan = "phucv";
string matkhau = "1234";
public FormDANG_NHAP()
{
    InitializeComponent();
}

private void pictureBox3_Click(object sender, EventArgs e)
{
}

private void label4_Click(object sender, EventArgs e)
{
}

private void DANG_NHAP_Load(object sender, EventArgs e)
{
}

private void btLogin_Click(object sender, EventArgs e)
{
    if (Kiemtrataikhoanmatkhau(txtUser.Text, txtPassword.Text))
    {
        FormLAM_VIEC f = new FormLAM_VIEC();
        f.Show();
        this.Hide();
    }
}
```

```
        else
        {
            MessageBox.Show("Check your account or password again", "Notification",
            MessageBoxButtons.OKCancel, MessageBoxIcon.Error);
            txtUser.Focus();
        }
    }
    bool Kiemtrataikhoanmatkhau(string tentaikhoan, string matkhau)
    {
        if (tentaikhoan == this.tentaikhoan && matkhau == this.matkhau)
        {
            return true;
        }
        return false;
    }

    private void txtUser_TextChanged(object sender, EventArgs e)
    {
    }
}
```

4.2.2. Control Interface

FormLAM_VIEC



XE VÀO

XE RA

02-05-2024 09:26:13



KẾT NỐI **NGẮT KẾT NỐI**

RFID 1

RFID 2

PIC

ID THẺ TỪ

ID

HỌ VÀ TÊN

THỜI GIAN

TRẠNG THÁI

BIỂN SỐ

LÊN **XUỐNG**

ĐĂNG XUẤT

CÓ RFID

STT	Số ID	Họ tên	Thời gian	Trạng thái	Biển số xe
1	407	Nhật	1	1	A700318
2	505	Phúc	1	1	B181904
3	486	Hào	1	1	A589053
4	1090	t	3	2	B4056
5	12435		01/05/2024 4:37:13	Ra	
6	407	Nhật	01/05/2024 5:52:46	Ra	A700318
7	407	Nhật	01/05/2024 5:56:00	Ra	A700318
8	407	Nhật	01/05/2024 5:56:05	Ra	A700318
9	407	Nhật	01/05/2024 5:56:17	Ra	A700318

CODE:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO.Ports;
using Emgu.CV;
using Emgu.CV.Structure;

```

```
using System.Data.OleDb;
```

```
using System.Drawing.Imaging;
```

```
namespace BAIDOXETUDONG
```

```
{
```

```
    public partial class FormLAM_VIEC : Form
```

```
    {
```

```
        bool isThoat = true;
```

```
        Capture _camera1;
```

```
        Capture _camera2;
```

```
        public FormLAM_VIEC()
```

```
        {
```

```
            InitializeComponent();
```

```
            timer3.Start();
```

```
            khoi_tao_bang();
```

```
            khoi_tao_bang1();
```

```
            loadData();
```

```
            loadData1();
```

```
        }
```

```
        private void loadData()
```

```
        {
```

```
            DataTable my_data = new DataTable();
```

```
            my_data.Load(Program.Database.SelectSQL("SELECT * FROM  
OINFO"));
```

```
            dataGridView1.DataSource = my_data;
```

```
            //Program.Database.Close();
```

```
        }
```

```
        private void khoi_tao_bang()
```

```
{
    dataGridView1.AutoGenerateColumns = false;
    const int NumberColumn = 7;
    dataGridView1.ColumnCount = NumberColumn;
    string[] list = new string[NumberColumn] { "STT", "ID", "Hovaten",
"thoigianvao", "thoigianra", "trangthai", "bienso" };
    string[] header_name = new string[NumberColumn] { "STT", "Số ID",
"Họ tên", "Thời gian vào", "Thời gian ra", "Trạng thái", "Biển số xe" };
    int[] width_col = new int[NumberColumn] { 50, 150, 150, 250, 250, 100,
150 };

    for (int i = 0; i < NumberColumn; i++)
    {
        dataGridView1.Columns[i].Name = list[i];
        dataGridView1.Columns[i].DataPropertyName = list[i];
        dataGridView1.Columns[i].HeaderText = header_name[i];
        dataGridView1.Columns[i].Width = width_col[i];
        dataGridView1.Columns[i].ValueType = typeof(String);
        dataGridView1.Columns[i].Visible = true;
    }
}

private void loadData1()
{
    DataTable my_data1 = new DataTable();
    my_data1.Load(Program.Database.SelectSQL("SELECT * FROM
DMD"));
    dataGridView2.DataSource = my_data1;
    //Program.Database.Close();
}
```

```
}  
private void khoi_tao_bang1()  
{  
    dataGridview2.AutoGenerateColumns = false;  
    const int NumberColumn = 4;  
    dataGridview2.ColumnCount = NumberColumn;  
    string[] list = new string[NumberColumn] { "STT", "ID", "Hovaten",  
    "Bienso" };  
    string[] header_name = new string[NumberColumn] { "STT", "Số ID",  
    "Họ tên", "Biển số xe" };  
    int[] width_col = new int[NumberColumn] { 50, 150, 150, 150 };  
  
    for (int i = 0; i < NumberColumn; i++)  
    {  
        dataGridview2.Columns[i].Name = list[i];  
        dataGridview2.Columns[i].DataPropertyName = list[i];  
        dataGridview2.Columns[i].HeaderText = header_name[i];  
        dataGridview2.Columns[i].Width = width_col[i];  
        dataGridview2.Columns[i].ValueType = typeof(String);  
        dataGridview2.Columns[i].Visible = true;  
    }  
}  
private void dataGridview1_CellContentClick(object sender,  
DataGridViewCellEventArgs e)  
{  
  
    int row = dataGridview1.CurrentCell.RowIndex;
```

```
        tbid.Text =
dataGridView1.Rows[row].Cells["ID"].FormattedValue.ToString();
        tbname.Text =
dataGridView1.Rows[row].Cells["Hovaten"].FormattedValue.ToString();
        tbTT.Text =
dataGridView1.Rows[row].Cells["trangthai"].FormattedValue.ToString();
        tbbienso.Text =
dataGridView1.Rows[row].Cells["bienso"].FormattedValue.ToString();
        tbtimevao.Text =
dataGridView1.Rows[row].Cells["thoigianvao"].FormattedValue.ToString();
        tbtimera.Text =
dataGridView1.Rows[row].Cells["thoigianra"].FormattedValue.ToString();

    }

    private void dataGridView2_CellContentClick(object sender,
DataGridViewCellEventArgs e)
    {

        int row = dataGridView2.CurrentCell.RowIndex;
        tbid.Text =
dataGridView2.Rows[row].Cells["ID"].FormattedValue.ToString();
        tbname.Text =
dataGridView2.Rows[row].Cells["Hovaten"].FormattedValue.ToString();
        //tbTT.Text =
dataGridView2.Rows[row].Cells["trangthai"].FormattedValue.ToString();
        tbbienso.Text =
dataGridView2.Rows[row].Cells["bienso"].FormattedValue.ToString();
```

```
//tbtimevao.Text =  
dataGridView2.Rows[row].Cells["thoigianvao"].FormattedValue.ToString();  
//tbtimera.Text =  
dataGridView2.Rows[row].Cells["thoigianra"].FormattedValue.ToString();  
  
}  
private void dataGridView2_RowsAdded(object sender,  
DataGridViewRowsAddedEventArgs e)  
{  
    for (int i = 0; i < dataGridView1.RowCount; i++)  
    {  
        dataGridView1.Rows[i].Cells[0].Value = Convert.ToString(i + 1);  
    }  
}  
  
private void dataGridView1_RowsAdded(object sender,  
DataGridViewRowsAddedEventArgs e)  
{  
    for (int i = 0; i < dataGridView1.RowCount; i++)  
    {  
        dataGridView1.Rows[i].Cells[0].Value = Convert.ToString(i + 1);  
    }  
}  
  
private void DOWN_Click(object sender, EventArgs e)  
{  
    serialPort3.Write("2");  
    //progressBar1.Value = 100;
```

```
}
```

```
private void btDANGXUAT_Click(object sender, EventArgs e)
```

```
{
```

```
    isThoat = false;
```

```
    this.Close();
```

```
    FormDANG_NHAP f = new FormDANG_NHAP();
```

```
    f.Show();
```

```
}
```

```
private void FormLAM_VIEC_FormClosed(object sender,  
FormClosedEventArgs e)
```

```
{
```

```
    if (isThoat)
```

```
        Application.Exit();
```

```
}
```

```
private void button2_Click(object sender, EventArgs e)
```

```
{
```

```
    //_camera1 = new Capture(0);
```

```
    //_camera1.Start();
```

```
    //_camera2 = new Capture(1);
```

```
    //_camera2.Start();
```

```
    //_camera1.ImageGrabbed += ProcessFrame1;
```

```
    //_camera2.ImageGrabbed += ProcessFrame2;
```

```
    try
```

```
    {
```

```
        //if (!serialPort1.IsOpen && !serialPort2.IsOpen &&
```

```
!serialPort3.IsOpen)
    //{
        serialPort1.PortName = CboxRF1.Text;
        serialPort1.Open();
        serialPort2.PortName = CboxRF2.Text;
        serialPort2.Open();
        serialPort3.PortName = Cboxpic.Text;
        serialPort3.Open();
        timer1.Start();
        timer4.Start();
        progressBar1.Value = 100;
        MessageBox.Show("DA KET NOI", "Thong bao",
MessageBoxButtons.OK, MessageBoxIcon.None);
    //}

}
catch (Exception Err)
{
    MessageBox.Show(Err.Message,"ERROR", MessageBoxButtons.OK,
MessageBoxIcon.None);
}
}
private void ProcessFrame1(object sender, EventArgs e)
{
    Mat frame = new Mat();
    _camera1.Retrieve(frame);
    CameraRA.Image = frame.ToImage<Bgr, byte>().Bitmap;
}
```

```
private void ProcessFrame2(object sender, EventArgs e)
{
    Mat frame = new Mat();
    _camera2.Retrieve(frame);
    CameraVao.Image = frame.ToImage<Bgr, byte>().Bitmap;
}

private void CameraVao_Click(object sender, EventArgs e)
{

}

private void UP_Click(object sender, EventArgs e)
{
    serialPort3.Write("1");
    //progressBar1.Value = 100;
}

private void FormLAM_VIEC_Load(object sender, EventArgs e)
{

}

private void comboBox2_SelectedIndexChanged(object sender, EventArgs
e)
{
```

```
}
```

```
private void comboBox3_SelectedIndexChanged(object sender, EventArgs e)
```

```
{
```

```
}
```

```
private void groupBox1_Enter(object sender, EventArgs e)
```

```
{
```

```
    string[] port = SerialPort.GetPortNames();
```

```
    CboxRF1.Items.AddRange(port);
```

```
    CboxRF2.Items.AddRange(port);
```

```
    Cboxpic.Items.AddRange(port);
```

```
}
```

```
private void progressBar1_Click(object sender, EventArgs e)
```

```
{
```

```
}
```

```
private void button2_Click_1(object sender, EventArgs e)
```

```
{
```

```
    serialPort1.Close();
```

```
    serialPort2.Close();
```

```
    serialPort3.Close();
```

```
    timer1.Stop();
```

```
    progressBar1.Value = 100;
```

```
    MessageBox.Show("DA NGAT KET NOI", "Thong bao",
```

```
MessageBoxButtons.OK, MessageBoxIcon.None);  
}
```

byte[] Bien_Bai1 = new byte[200]; // timer kích hoạt sau mỗi khoảng thời gian do một Timer gửi tín hiệu. Đây là một mảng byte có kích thước 200 byte

//Trong C#, một mảng byte như vậy là một mảng các giá trị byte có thể lưu trữ tối đa 200 byte. Các phần tử của mảng này có thể lưu trữ các giá trị byte từ 0 đến 255.

```
int numByte;  
public static class GlobalData  
{  
    public static string SharedString { get; set; } = "Initial value";  
    public static string time1 { get; set; } = "Initial value";  
    public static string time2 { get; set; } = "Initial value";  
}  
private void timer1_Tick(object sender, EventArgs e)  
{  
    byte[] temp = new byte[12]; // Một mảng byte tạm thời với kích thước 12  
byte.  
    if (serialPort2.IsOpen)  
    {  
        numByte = serialPort2.BytesToRead; //được sử dụng để lấy số lượng  
byte đang chờ để đọc từ cổng serial được mở  
        //Giá trị trả về từ BytesToRead là số lượng byte còn lại trong bộ đệm  
đọc của cổng serial. Bạn có thể sử dụng giá trị này để biết khi nào cần đọc dữ  
liệu từ cổng serial, tránh việc đọc dữ liệu khi không có dữ liệu mới.
```

```
        //Sau khi bạn đã lấy số lượng byte cần đọc(numByte), bạn có thể sử  
dụng phương thức Read để đọc các byte từ cổng serial và lưu vào một mảng
```

byte như trong đoạn mã bạn đã cung cấp.

```
        if (numByte > 0)
        {
            serialPort2.Read(Bien_Bai1, 0, numByte); //Dòng mã
serialPort2.Read(Bien_Bai1, 0, numByte); trong chương trình của bạn được sử
dụng để đọc dữ liệu từ cổng serial đã mở (serialPort2) và lưu trữ vào mảng byte
Bien_Bai1.numByte: là số lượng byte cần đọc từ cổng serial.

            for (int k = 1; k < 13; k++)
            {
                temp[k - 1] = Bien_Bai1[k];
            }
            string idNumber = BitConverter.ToString(temp); // chuyển chuỗi
byte thành string gắn vô id
            idNumber = string.Join("", idNumber.Split('-')); // bỏ dấu trừ giữa
các byte

                                                    // đưa mảng về đúng số int
            tbidthetu.Text = idNumber;
            //string DayTime = DateTime.Now.ToString("dd-MM-yyyy-hh-
mm-ss");
            //string filename = @"C:\Users\Dell\OneDrive - The University of
Technology\Máy tính\anh\" + DayTime + ".png";
            //CameraVao.Image.Save(filename, ImageFormat.Png);
            pictureBox3.Image = CameraVao.Image;

            if (idNumber == "334330303045463545383246")
            {
                string ten =
dataGridView2.Rows[0].Cells["Hovaten"].Value.ToString();
                tbname.Text = ten;
```

```
        string id =
dataGridView2.Rows[0].Cells["ID"].Value.ToString();
        tbid.Text = id;
        string bienso =
dataGridView2.Rows[0].Cells["bienso"].Value.ToString();
        tbbienso.Text = bienso;
        tbTT.Text = "Ra";
        string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");

        tbtimera.Text = NgayGio;
        tbtimevao.Text = GlobalData.SharedString;
        //String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra , trangthai , bienso) values ("
        //      + tbid.Text + ","
        //      + tbname.Text + ","
        //      + GlobalData.SharedString + ","
        //      + NgayGio + ","
        //      + tbTT.Text + ","
        //      + tbbienso.Text + ")";
        //OleDbDataReader reader = Program.Database.SelectSQL(sql);
        //loadData();
    }
    if (idNumber == "334330303045444642363542")
    {
        string ten =
dataGridView2.Rows[1].Cells["Hovaten"].Value.ToString();
```

```
tbname.Text = ten;

string id =
dataGridView2.Rows[1].Cells["ID"].Value.ToString();
tbid.Text = id;
string bienso =
dataGridView2.Rows[1].Cells["bienso"].Value.ToString();
tbbienso.Text = bienso;
tbTT.Text = "Ra";
string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");
tbtimera.Text = NgayGio;
tbtimevao.Text = GlobalData.time1;
//String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra, trangthai, bienso) values ("
//      + tbid.Text + ","
//      + tbname.Text + ","
//      + GlobalData.time1 + ","
//      + NgayGio + ","
//      + tbTT.Text + ","
//      + tbbienso.Text + ")";
//OleDbDataReader reader = Program.Database.SelectSQL(sql);
//loadData();
}
if (idNumber == "334330303045464638363442")
{
    string ten =
dataGridView2.Rows[2].Cells["Hovaten"].Value.ToString();
    tbname.Text = ten;
```

```
        string id =
dataGridView2.Rows[2].Cells["ID"].Value.ToString();
        tbid.Text = id;
        string bienso =
dataGridView2.Rows[2].Cells["bienso"].Value.ToString();
        tbbienso.Text = bienso;
        tbTT.Text = "Ra";
        string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");
        GlobalData.SharedString = NgayGio;
        tbtimevao.Text = NgayGio;
        tbtimera.Text = GlobalData.time2;
        //String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra, trangthai , bienso) values ("
        //      + tbid.Text + ","
        //      + tbname.Text + ","
        //      + NgayGio + ","
        //      + tbtimera.Text + ","
        //      + tbTT.Text + ","
        //      + tbbienso.Text + ")";
        //OleDbDataReader reader = Program.Database.SelectSQL(sql);
        //loadData();
    }

    serialPort3.Write("1");
    timer2.Start();
    timer5.Start();
}
```

```
    }  
  
}
```

```
private void timer2_Tick(object sender, EventArgs e)  
{  
    serialPort3.Write("2");  
}
```

```
private void timer3_Tick(object sender, EventArgs e)  
{  
    label4.Text = DateTime.Now.ToString("dd-MM-yyyy hh:mm:ss tt");  
}  
byte[] Bien_Bai2 = new byte[200];  
int numByte1 ;  
private void timer4_Tick(object sender, EventArgs e)  
{  
    byte[] temp = new byte[12]; // Một mảng byte tạm thời với kích thước 12  
byte.  
    if (serialPort1.IsOpen)  
    {  
        numByte1 = serialPort1.BytesToRead; // được sử dụng để lấy số lượng  
byte đang chờ để đọc từ cổng serial được mở  
        //Giá trị trả về từ BytesToRead là số lượng byte còn lại trong bộ đệm  
đọc của cổng serial. Bạn có thể sử dụng giá trị này để biết khi nào cần đọc dữ  
liệu từ cổng serial, tránh việc đọc dữ liệu khi không có dữ liệu mới.
```

//Sau khi bạn đã lấy số lượng byte cần đọc(numByte), bạn có thể sử dụng phương thức Read để đọc các byte từ cổng serial và lưu vào một mảng byte như trong đoạn mã bạn đã cung cấp.

```
if (numByte1 > 0)
```

```
{
```

```
    serialPort1.Read(Bien_Bai2, 0, numByte1); //Dòng mã
```

serialPort2.Read(Bien_Bai1, 0, numByte); trong chương trình của bạn được sử dụng để đọc dữ liệu từ cổng serial đã mở (serialPort2) và lưu trữ vào mảng byte Bien_Bai1.numByte: là số lượng byte cần đọc từ cổng serial.

```
    for (int k = 1; k < 13; k++)
```

```
    {
```

```
        temp[k - 1] = Bien_Bai2[k];
```

```
    }
```

```
    string idNumber = BitConverter.ToString(temp); // chuyển chuỗi
```

byte thành string gắn vô id

```
    idNumber = string.Join("", idNumber.Split('-')); // bỏ dấu trừ giữa
```

các byte

```
    // đưa mảng về đúng số int
```

```
    tbidthetu.Text = idNumber;
```

```
    //string DayTime = DateTime.Now.ToString("dd-MM-yyyy-hh-mm-ss");
```

```
    //string filename = @"C:\Users\Dell\OneDrive - The University of Technology\Máy tính\anh\" + DayTime + ".png";
```

```
    //CameraRA.Image.Save(filename, ImageFormat.Png);
```

```
    pictureBox4.Image = CameraVao.Image;
```

```
    if (idNumber == "334330303045463545383246")
```

```
{
```

```
        string ten =
dataGridView2.Rows[0].Cells["Hovaten"].Value.ToString();
        tbname.Text = ten;

        string id =
dataGridView2.Rows[0].Cells["ID"].Value.ToString();
        tbid.Text = id;
        string bienso =
dataGridView2.Rows[0].Cells["bienso"].Value.ToString();
        tbbienso.Text = bienso;
        tbTT.Text = "Vào";
        string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");
        GlobalData.SharedString = NgayGio;
        tbtimevao.Text = NgayGio;
        tbtimera.Text = "---";
        //String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra, trangthai , bienso) values ("
        //      + tbid.Text + ","
        //      + tbname.Text + ","
        //      + NgayGio + ","
        //      + tbtimera.Text + ","
        //      + tbTT.Text + ","
        //      + tbbienso.Text + ")";
        //OleDbDataReader reader = Program.Database.SelectSQL(sql);
        //loadData();
    }
    if (idNumber == "334330303045444642363542")
    {
```

```
        string ten =
dataGridView2.Rows[1].Cells["Hovaten"].Value.ToString();
        tbname.Text = ten;

        string id =
dataGridView2.Rows[1].Cells["ID"].Value.ToString();
        tbid.Text = id;
        string bienso =
dataGridView2.Rows[1].Cells["bienso"].Value.ToString();
        tbbienso.Text = bienso;
        tbTT.Text = "Vào";
        string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");
        GlobalData.time1 = NgayGio;
        tbtimevao.Text = NgayGio;
        tbtimera.Text = "---";
        //String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra , trangthai , bienso) values ("
        //      + tbid.Text + ","
        //      + tbname.Text + ","
        //      + NgayGio + ","
        //      + tbtimera.Text + ","
        //      + tbTT.Text + ","
        //      + tbbienso.Text + ")";
        //OleDbDataReader reader = Program.Database.SelectSQL(sql);
        //loadData();
    }
    if (idNumber == "334330303045464638363442")
    {
```

```
        string ten =
dataGridView2.Rows[2].Cells["Hovaten"].Value.ToString();
        tbname.Text = ten;

        string id =
dataGridView2.Rows[2].Cells["ID"].Value.ToString();
        tbid.Text = id;
        string bienso =
dataGridView2.Rows[2].Cells["bienso"].Value.ToString();
        tbbienso.Text = bienso;
        tbTT.Text = "Vào";
        string NgayGio = DateTime.Now.ToString("dd/MM/yyyy
h:mm:ss");
        GlobalData.time2 = NgayGio;
        tbtimevao.Text = NgayGio;
        tbtimera.Text = "---";
        //String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra, trangthai , bienso) values ("
        //      + tbid.Text + ","
        //      + tbname.Text + ","
        //      + NgayGio + ","
        //      + tbtimera.Text + ","
        //      + tbTT.Text + ","
        //      + tbbienso.Text + ")";
        //OleDbDataReader reader = Program.Database.SelectSQL(sql);
        //loadData();
    }
    serialPort3.Write("1");
    timer2.Start();
```

```
    }

    }

}

private void button3_Click(object sender, EventArgs e)
{
    string ten = dataGridView2.Rows[1].Cells["Hovaten"].Value.ToString();
    tbname.Text = ten;

    string id = dataGridView2.Rows[1].Cells["ID"].Value.ToString();
    tbid.Text = id;
    string bienso =
dataGridView2.Rows[1].Cells["bienso"].Value.ToString();
    tbbienso.Text = bienso;
    tbTT.Text = "Ra";

    string NgayGio = DateTime.Now.ToString("dd/MM/yyyy h:mm:ss");

    tbtimera.Text = NgayGio;
    timer5.Start();
    //String sql = "insert into OINFO (ID , Hovaten , thoigianvao, thoigianra,
trangthai , bienso) values ("
    //      + tbid.Text + ","
    //      + tbname.Text + ","
    //      + GlobalData.SharedString + ","
    //      + tbtimera.Text + ","
    //      + tbTT.Text + ","
    //      + tbbienso.Text + ")";
```

```
//OleDbDataReader reader = Program.Database.SelectSQL(sql);
//loadData();
}

private void timer5_Tick(object sender, EventArgs e)
{
    String sql = "insert into OINFO (ID , Hovaten , thoigianvao, thoigianra,
trangthai , bienso) values ("
        + tbid.Text + ","
        + tbname.Text + ","
        + GlobalData.SharedString + ","
        + tbtimera.Text + ","
        + tbTT.Text + ","
        + tbbienso.Text + ")";
    OleDbDataReader reader = Program.Database.SelectSQL(sql);
    loadData();
    timer5.Stop();
}

private void button1_Click_1(object sender, EventArgs e)
{
    string ten = dataGridView2.Rows[1].Cells["Hovaten"].Value.ToString();
    tbname.Text = ten;

    string id = dataGridView2.Rows[1].Cells["ID"].Value.ToString();
    tbid.Text = id;
    string bienso =
dataGridView2.Rows[1].Cells["bienso"].Value.ToString();
    tbbienso.Text = bienso;
```

```

tbTT.Text = "Vào";

string NgayGio = DateTime.Now.ToString("dd/MM/yyyy h:mm:ss");
GlobalData.SharedString = NgayGio;
tbtimevao.Text = NgayGio;
tbtimera.Text = "---";
timer5.Start();

// String sql = "insert into OINFO (ID , Hovaten , thoigianvao,
thoigianra, trangthai , bienso) values ("
//      + tbid.Text + ","
//      + tbname.Text + ","
//      + NgayGio + ","
//      + tbtimera.Text + ","
//      + tbTT.Text + ","
//      + tbbienso.Text + ")";

//OleDbDataReader reader = Program.Database.SelectSQL(sql);
//loadData();
}
}
}

```

4.3. Instructions for use:

4.3.1. Hardware Instructions:

- Step 1: Connect power to the power supply circuit to provide power to the PIC circuit.
- Step 2: Connect all ports such as Camera, RFID, UART to the computer.

4.3.2. Software Instructions:

- Step 1: Select appropriate COM ports for the Camera and RFID.
- Step 2: Press Connect to establish the connections.

- Step 3: After establishing the connections, start scanning the card and guide the vehicle to the designated parking spot with the sensor.

CHAPTER 5: CONCLUSION

RESULTS OF IMPLEMENTATION

5.1. Achievements

- Design and complete the mechanical model of the color grading system
- Researching and applying PIC 16F877A microcontroller as a microcontroller for the system
- Complete all the hardware for the system to work as required
- System power circuit design
- Design central control circuit using microcontroller PIC 16F877A
- Display circuit design
- Assemble the circuit according to the requirements to ensure the technicality and aesthetics

5.2. Development direction of the topic

Current parking lots have been integrated with many advanced technologies to help operate and operate effectively and stably. During the time of researching the topic as well as participating in the actual construction of a real system, I proposed some directions for the development of the topic:

- Collecting fees automatically through license plate recognition, fees are deducted directly from the customer's account that has been registered in advance, completely without using the barrier system, magnetic card.
- Accessing the location of customers' vehicles, improving the ability of the parking system to guide the parking lot through arranging the structure of parking spaces.
- Developing the topic of applying future technologies such as: Detecting fraudulent vehicles, theft, and wanted suspects based on data provided by authorities

REFERENCES

REFERENC SUBJECTS

Programming book for microcontrollers and peripherals by teacher Dang Phuoc Vinh.

WEBSITE REFERENCES

<https://www.studocu.com/vn/document/truong-dai-hoc-giao-thong-van-tai/bai-do-xe-thong-minh/f23-do-an-bai-do-xe-thong-minh/60330625?origin=home-recent-1>